Agentic Screenwriting Studio

The Agentic Screenwriting Studio is a multi-agent AI system designed to act as a virtual writers' room. It helps screenwriters take an idea from a simple one-line pitch to a full, continuity-checked screenplay.

Under the hood, it uses the Google Agent Development Kit (ADK) to orchestrate a team of specialized AI agents powered by Gemini, each handling a specific part of the screenwriting process.

1. Architecture & Tech Stack

The project is split into a modern frontend and an AI-driven backend:
- Frontend (Next.js 14, TypeScript): Provides the UI for the "Writers' Room." It connects to the backend via standard HTTP REST APIs for actions (like creating projects) and WebSockets for real-time agent updates.
- Backend (Python, FastAPI): The core engine that runs the ADK runner, manages the database, and exposes the API endpoints.
- AI Models: Uses Gemini 2.5 (Flash/Pro) for text generation, Imagen 3 for generating concept art, and Gemini TTS for multi-speaker audio generation.
- Storage (SQLite + ChromaDB):
  - SQLite is used to persistently store the structured ScriptState (projects, scenes, characters, beat sheets).
  - ChromaDB is used as a vector database for Retrieval-Augmented Generation (RAG). Every scene and fact is indexed here so the system can quickly search for continuity conflicts.

2. The AI Agents (The Writers' Room)

The system operates using a "Coordinator and Workers" pattern.

- Showrunner (Coordinator): This is the main ADK agent that the user talks to. The Showrunner understands the user's request, decides which specialist agents are needed, routes the tasks to them, and enforces quality gates (like making sure a scene passes continuity checks before saving it).
- Story Architect: Generates structured beat sheets and story arcs from the user's raw pitch using established story frameworks.
- Dialogue Specialist: Takes the beat sheet and the Character Bible and drafts full scenes with authentic character dialogue.
- Continuity Checker: This agent uses ChromaDB to perform vector searches against all previously written scenes. It ensures that new scenes don't break established canon (e.g., a character can't hold a gun if they dropped it in the previous scene).
- Research Agent: Uses a Parallel Search API to perform live fact-checking on the web, allowing writers to

verify historical or technical details instantly.
- Rights & Clearance: Analyzes the script to flag potential legal, copyright, or clearance risks.
- Visualizer: Uses Imagen 3 to generate mood boards and concept art based on the script's locations and tone.
- Table Read: Generates multi-speaker audio performances of the written scenes using Gemini TTS.

3. The Core Workflow (How a Script Gets Written)

Here is the step-by-step flow of data when a user interacts with the app:

1. Pitching: The user types a request into the frontend chat (e.g., "Write a scene where the detective discovers his partner is corrupt").
2. API & WebSocket Routing: The frontend hits the /api/chat FastAPI endpoint. The backend initializes an ADK Runner session and passes the message to the Showrunner. Concurrently, a WebSocket connection (/ws/events) is used to stream the agent's real-time status and text back to the UI.
3. Delegation: The Showrunner analyzes the request and triggers a tool to call the Dialogue Specialist to draft the scene.
4. Forced Continuity Gate: Once the scene is drafted, the Showrunner is programmed not to finalize it immediately. It must first pass it to the Continuity Checker. The Checker queries ChromaDB, validates the scene against past facts, and either approves it or kicks it back for revisions.
5. State Update: Once approved, the backend updates the ScriptState in the SQLite database and adds the new scene embeddings to ChromaDB.
6. Export: At any point, the user can export the persistent state into industry-standard formats like .fountain or plain text, via the /api/script/{id}/export endpoint.